

Documentação

O sistema descrito nesta documentação pode ser encontrado no seguinte repositório do GitHub: <https://github.com/robertotomchak/seu-cantinho>.

Colaboradores

- Igor Neves da Silva (GRR20232377)
- Ricardo Prado Faria (GRR20235161)
- Roberto Sprengel Minozzo Tomchak (GRR20232369)

Funcionalidades Essenciais

1. Cadastro dos espaços disponíveis para aluguel, com informações detalhadas como fotos, capacidade e preço;
2. Gerenciamento das reservas futuras, contemplando data, cliente, espaço alugado e valor pago;
3. Controle de pagamentos, indicando se a reserva possui apenas sinal pago ou se já está quitada.

Requisitos de Qualidade

A proprietária da rede, Dona Maria, possuía uma solução antiga, desenvolvida de forma emergencial para atender demandas iniciais. Nessa solução, havia graves problemas de *double-booking*, ou seja, o mesmo espaço poderia ser reservado simultaneamente por diversos clientes. Outros problemas citados nessa antiga aplicação eram perda de informações financeiras e alto tempo de operação.

Além disso, a necessidade de uma nova solução se dá pelo crescimento da empresa, que expandiu para três estados diferentes, e assim o sistema é insuficiente para lidar com as novas demandas. Dona Maria também deseja que a nova aplicação seja simples, intuitiva e tolerante à falhas, devido à sua baixa familiaridade com tecnologias.

Com essas informações em mente, definimos que os seguintes requisitos de qualidade devem ser priorizados, em ordem do mais relevante para o menos:

1. **Confiabilidade:** a maior necessidade do sistema é evitar problemas de consistência, como *double-booking*;
2. **Simplicidade:** devido à baixa familiaridade com tecnologia, a proprietária da rede deseja um sistema simples. Por essa ser uma característica descrita bem explicitamente no enunciado, julgamos ser de alta importância;
3. **Usabilidade:** é necessário que o sistema seja fácil de usar, uma vez que usuários sem alto conhecimento técnico irão usá-lo;

4. **Escalabilidade:** a nova aplicação se tornou ineficiente em grande parte por sua baixa escalabilidade, e é plausível esperar que a rede irá se expandir novamente. Assim, é preciso que o sistema seja escalável. Entretanto, não queremos que a escalabilidade venha ao custo de um sistema mais simples e confiável;
5. **Manutenibilidade:** além de ajudar a garantir confiabilidade, problemas de manutenção podem prejudicar o funcionamento, como o sistema entrar em manutenção por muito tempo e um usuário não poder se comunicar com o dono do espaço da festa, por exemplo;
6. **Desempenho:** embora alto tempo de operação seja um requisito desejável, ele entra em conflito com outros requisitos, como confiabilidade, que possuem maior prioridade.

Estilo Arquitetural

Levando em consideração os requisitos e atributos de qualidade desejados, o estilo arquitetural escolhido para essa aplicação é o Cliente-Servidor com interface HTTP/REST. Com esse estilo, podemos separar bem a interface do cliente com o servidor, garantindo alta confiabilidade e bom desempenho, além de deixar bem claro a separação de responsabilidades. Essa separação entre cliente e servidor também torna o sistema mais fácil de ser usado, uma vez que o cliente não entra em contato direto com o servidor. O servidor será organizado usando Arquitetura em Camadas, uma vez que é uma solução simples de entender e garante boa confiabilidade. Dessa forma, atingimos um sistema de alta confiabilidade, simples de ser compreendido pela Dona Maria e que ainda garante boa escalabilidade e manutenibilidade.

O servidor em camadas está organizado em três camadas. A camada mais acima, o Controller, se comunica com o cliente, e envia requisições para a camada abaixo. Abaixo dele temos a camada de Serviço, a qual realiza os quatro serviços principais do sistema: cadastro de espaços, gerenciamento de reservas, login/cadastro de usuários e controle de pagamentos. Abaixo dela, temos a camada de Repositório, o qual se comunica com o banco de dados, que possui informações sobre os espaços e reservas do sistema, bem como os usuários cadastrados. Com isso, cada camada possui responsabilidades claras, garantindo alta coesão.

Modelagem UML

Diagrama de Classes

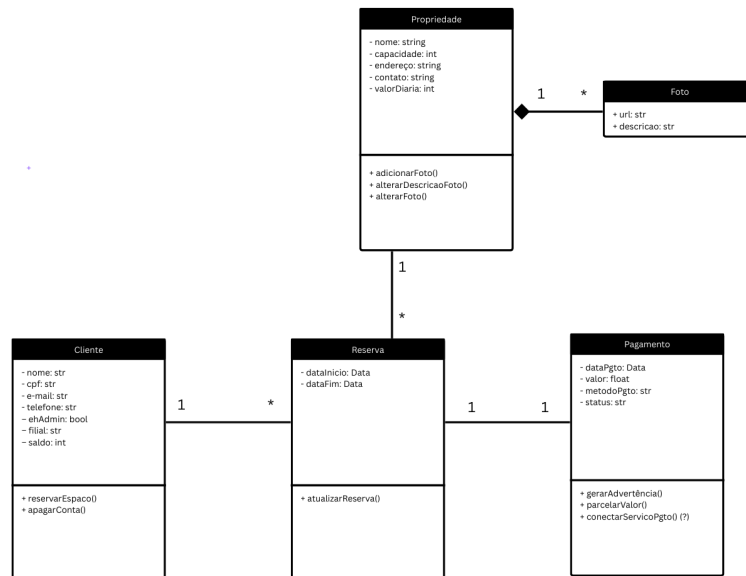
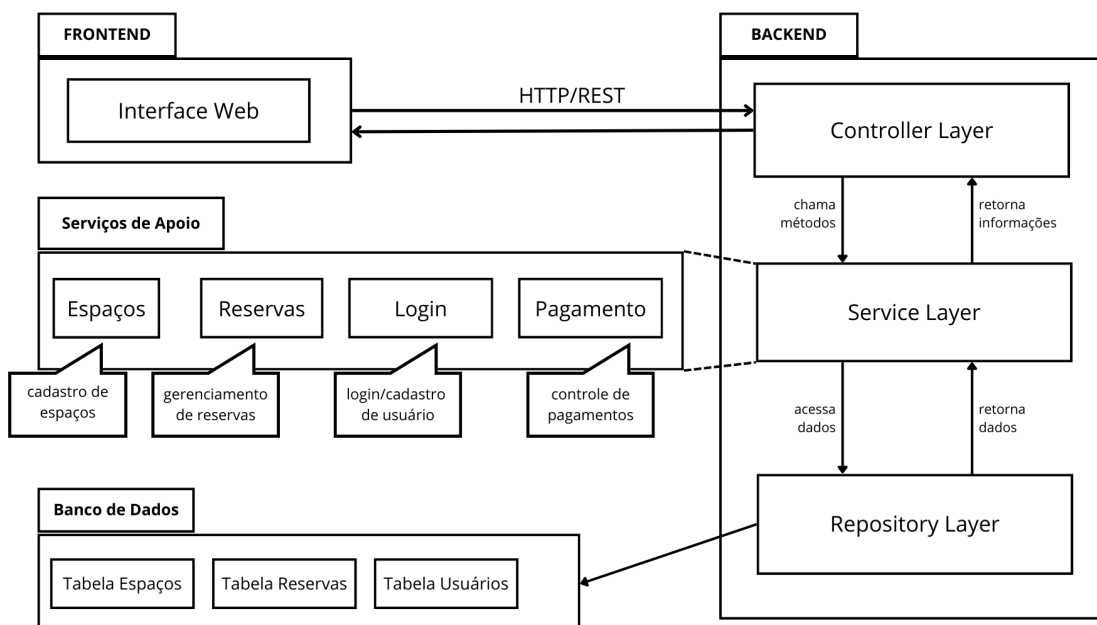


Diagrama de Componentes



Tecnologias Utilizadas

Para esse sistema, foi utilizado o Node Js (versão 14.0.0) para o frontend e a linguagem Python (versão 3.11.13) para o backend. Essas tecnologias foram escolhidas devido a sua simplicidade e velocidade de desenvolvimento para aplicações simples. Para o banco de dados, usamos MySQL (versão 8.0). Além disso, para que o trabalho possa facilmente ser colocado em produção e ser executado em diferentes ambientes, usamos Docker (versão 28.4.0).

Mapeamento UML -> Código

A estrutura do código segue a seguinte árvore:

```
├── docker
│   ├── backend.Dockerfile
│   └── frontend.Dockerfile
├── docker-compose.yml
├── LICENSE
├── README.md
├── src
│   ├── backEnd
│   │   ├── controller.py
│   │   ├── db.py
│   │   ├── models.py
│   │   ├── requirements.txt
│   │   └── service.py
│   ├── bd
│   │   └── scriptStart.sql
│   └── frontEnd
│       ├── package.json
│       ├── package-lock.json
│       ├── pages
│       │   └── index.js
│       ├── public
│       │   ├── homePage.html
│       │   ├── img
│       │   │   └── logo.png
│       │   ├── scripts
│       │   │   ├── listeners.js
│       │   │   ├── logic.js
│       │   │   └── scripts.js
│       └── style.css
```

Dentro do diretório `src/`, podemos claramente ver a separação entre frontend e backend em dois diretórios (`frontEnd/` e `backEnd/`, respectivamente). A arquitetura em camadas do backend está separada em três códigos-fonte, cada um representando a sua respectiva camada: [controller.py](#) para o controller, [service.py](#) para o service e [db.py](#) para o repository. O banco de dados está localizado em `src/bd/scriptStart.sql`, e segue o descrito no diagrama de classes, contendo tabelas para clientes, propriedades, reservas e pagamentos. Os códigos fonte estão bem organizados e documentados, com as funções organizadas de acordo com o serviço.

O uso de Docker no sistema se faz de duas formas. O arquivo `docker-compose.yml` é usado para colocar o sistema como um todo num contêiner, para que seja levantado em qualquer sistema. Além disso, no diretório `docker/`, temos os arquivos `frontend.Dockerfile` e `backend.Dockerfile` que criam os contêineres do frontend e backend, respectivamente. Com isso, permitimos que cada uma dessas partes execute independentemente. No backend, a lógica do código garante que as camadas apenas se comuniquem com as camadas adjacentes. Por exemplo, a camada de controller ([controller.py](#)) apenas inclui as funções da camada de serviço (`service.py`).

Os arquivos e diretórios restantes garantem a funcionalidade do git e a documentação. No arquivo `openapi.json`, temos a documentação da API REST em formato OpenAPI/Swagger.

Para garantir a confiabilidade do sistema e evitar problemas como *double-booking*, são usados locks ao realizar operações no banco de dados que podem gerar condições de disputa.

Operações CRUD

Para o sistema foram desenvolvidas operações CRUD (Create, Read, Update, Delete) para usuários, propriedades, reservas e pagamentos. Entretanto, na nossa implementação julgamos por limitar algumas operações, as quais estão descritas a seguir.

Pagamentos: possui apenas a operação Create. Isso porque uma operação bancária não deve ser alterada após realizada, pois pode causar conflito no saldo dos envolvidos, além de gerar possíveis fraudes. Além disso, um usuário que não tenha acesso ao banco não deve ser capaz de ler um pagamento.

Reservas: possui todas as operações, exceto a Update. Julgamos que, neste protótipo inicial, atualizações nas reservas criariam conflitos complexos de serem resolvidos no pagamento delas, uma vez que impedimos a atualização de pagamentos, conforme descrito acima.

Como usar

Todas as instruções de uso do sistema podem ser encontradas no arquivo [README.md](#).

Reflexões Finais

Com o sistema completo, concluímos que a escolha de um sistema Cliente-Servidor com backend em camadas foi muito benéfico, uma vez que permitiu uma organização muito simples do sistema, facilitando para adicionar funcionalidades aos poucos nele. Com isso, é simples adicionar novas funcionalidades no futuro, tornando o sistema bem escalável.

A maior dificuldade apresentada durante o trabalho foi a baixa familiaridade da equipe com o uso de Dockers. Devido à isso, ao longo do desenvolvimento foi complicado garantir que todos os códigos estivessem funcionando adequadamente dentro de seus contêineres. Entretanto, uma vez que compreendemos bem o funcionamento do Docker, levantar os contêineres para realizar testes foi simples.

Acreditamos que todos os requisitos de qualidade foram satisfeitos, em especial a confiabilidade e a simplicidade do sistema. Entretanto, nos próximos passos poderia ser melhorado a usabilidade do sistema, uma vez que a interface do frontend está muito simples, e o código do service layer pode ser refatorado para garantir uma melhor manutenibilidade.